

Customer Churn Prediction Using Neural Networks and TensorFlow

A Deep Learning Approach for Bank Customer Retention Analysis

Abstract

Customer churn — the loss of clients to competitors — is a critical business problem in the banking sector. Predicting which customers are likely to leave allows institutions to take proactive retention measures. This project builds a Customer Churn Prediction System using an Artificial Neural Network (ANN) implemented with TensorFlow/Keras, trained on the Bank Customer Churn dataset. The model uses three hidden layers with progressive Dropout regularization and Early Stopping to prevent overfitting. A custom prediction threshold of **0.35** is applied to improve recall for the minority churn class. On a held-out test set of 2,000 samples, the model achieves an overall accuracy of **86%**, with a weighted F1-score of **0.85** and strong precision/recall for the non-churn class.

Keywords: Neural Networks, TensorFlow, Customer Churn, Binary Classification, Dropout Regularization, Early Stopping, Bank Analytics

1. Introduction

Customer churn prediction is a well-studied problem in the financial industry. Banks invest significant resources in acquiring customers, making it far more cost-effective to retain existing ones than to acquire new ones. Identifying customers who are likely to leave — before they do — gives banks the opportunity to intervene with targeted offers or improved service.

Deep Learning and Artificial Neural Networks have proven highly effective for tabular classification tasks in financial analytics. This project applies a multi-layer ANN to the Churn Modelling dataset, incorporating best practices such as feature encoding, feature scaling, Dropout regularization, and Early Stopping for robust model training.

2. Problem Statement

The objective is to build a binary classification model that predicts whether a bank customer will churn (leave the bank) based on their demographic and account features.

Objectives

- Load and explore the Bank Customer Churn dataset.
- Preprocess features through encoding and scaling.
- Build a multi-layer ANN with Dropout regularization using TensorFlow.
- Train with Early Stopping to prevent overfitting.

- Evaluate the model using accuracy, precision, recall, and F1-score.
- Tune the prediction threshold to improve churn recall.

3. Dataset Description

The dataset used is the **Churn_Modelling.csv** file, a widely used bank customer churn benchmark dataset. It contains 10,000 customer records with 14 original columns, of which 3 non-informative columns are dropped before training.

Features Used After Preprocessing

Feature	Type	Description
CreditScore	Numerical	Customer's credit score
Gender	Encoded	Male / Female → 0 / 1 via LabelEncoder
Age	Numerical	Age of the customer
Tenure	Numerical	Years as a bank customer
Balance	Numerical	Account balance
NumOfProducts	Numerical	Number of bank products used
HasCrCard	Numerical	Has credit card (0 or 1)
IsActiveMember	Numerical	Is an active member (0 or 1)
EstimatedSalary	Numerical	Estimated annual salary
Geography_Germany	One-Hot	Customer from Germany (drop_first=True)
Geography_Spain	One-Hot	Customer from Spain
Exited (Target)	Binary	0 = Stayed, 1 = Churned

Dropped Columns

- RowNumber — Row index, non-informative
- CustomerId — Unique ID, non-informative
- Surname — Name, non-informative

Class Distribution

The dataset is imbalanced: approximately **79.6%** of customers stayed (class 0) and only **20.4%** churned (class 1). This imbalance is handled at inference time via a lowered prediction threshold of 0.35 to improve recall for the minority churn class.

4. Tools and Technologies

- Python
- TensorFlow / Keras
- Scikit-Learn
- Pandas
- NumPy
- Matplotlib
- Jupyter Notebook

5. Methodology

Step 1 — Data Loading

The dataset is loaded using Pandas from a CSV file:

```
df = pd.read_csv('Churn_Modelling.csv')
```

Step 2 — Data Exploration

- Shape and data types inspected with `df.shape` and `df.info()`
- Missing values checked using `df.isnull().sum()`
- Statistical summary reviewed to understand feature distributions

Step 3 — Preprocessing

- **Drop columns:** RowNumber, CustomerId, Surname removed as non-informative.
- **Gender encoding:** LabelEncoder converts Gender to binary (0/1).
- **Geography encoding:** `pd.get_dummies` with `drop_first=True` creates two indicator columns (Geography_Germany, Geography_Spain), with France as the baseline.
- **Feature scaling:** StandardScaler applied — fit on training set, transform on both training and test sets to prevent data leakage.

Step 4 — Train-Test Split

- Training set: 80% (8,000 samples)
- Test set: 20% (2,000 samples)
- `random_state=42` for reproducibility

Step 5 — Prediction Threshold Tuning

Rather than the default threshold of 0.50, predictions are binarized at **0.35** (`pred > 0.35`). This lowers the bar for predicting churn, increasing recall for the minority class (churned customers) at the cost of some precision.

6. Neural Network Architecture

The ANN uses three hidden Dense layers with progressively decreasing width and decreasing Dropout rates, followed by a sigmoid output layer for binary classification.

Layer	Units / Rate	Activation
Hidden Layer 1 (Dense)	64 Neurons	ReLU
Dropout 1	Rate = 0.3	—
Hidden Layer 2 (Dense)	32 Neurons	ReLU
Dropout 2	Rate = 0.2	—
Hidden Layer 3 (Dense)	16 Neurons	ReLU
Dropout 3	Rate = 0.1	—
Output Layer (Dense)	1 Neuron	Sigmoid

Architecture Summary:

```
Input → Dense(64,ReLU) → Dropout(0.3) → Dense(32,ReLU) → Dropout(0.2) → Dense(16,ReLU) → Dropout(0.1) → Dense(1,Sigmoid)
```

The decreasing Dropout schedule (0.3 → 0.2 → 0.1) applies stronger regularization to the wider early layers where overfitting risk is highest, and lighter regularization near the output where the network refines its decision boundary.

7. Model Training

- **Optimizer:** Adam
- **Loss Function:** Binary Crossentropy
- **Metric:** Accuracy
- **Epochs:** up to 50 (Early Stopping applied)
- **Batch Size:** 32
- **Validation Split:** 20% of training data

Early Stopping

An EarlyStopping callback monitors **val_loss** with patience=5. Training halts if validation loss does not improve for 5 consecutive epochs, and the best weights are restored automatically (restore_best_weights=True). Training converged at approximately epoch 20.

8. Results and Evaluation

The model is evaluated on the held-out test set of **2,000 samples** (1,607 non-churn, 393 churn).

Overall Test Accuracy: 86%

Classification Report

Class	Precision	Recall	F1-Score	Support
0 (Stayed)	0.90	0.92	0.91	1,607
1 (Churned)	0.64	0.59	0.62	393
Accuracy			0.86	2,000
Macro Avg	0.77	0.76	0.76	2,000
Weighted Avg	0.85	0.86	0.85	2,000

Confusion Matrix

Of 1,607 customers who stayed, 1,479 were correctly predicted (TN) and 128 were false positives. Of 393 churned customers, 232 were correctly detected (TP) and 161 were missed (FN).

	Predicted: Stay	Predicted: Churn
Actual: Stay	1,479 (TN)	128 (FP)
Actual: Churn	161 (FN)	232 (TP)

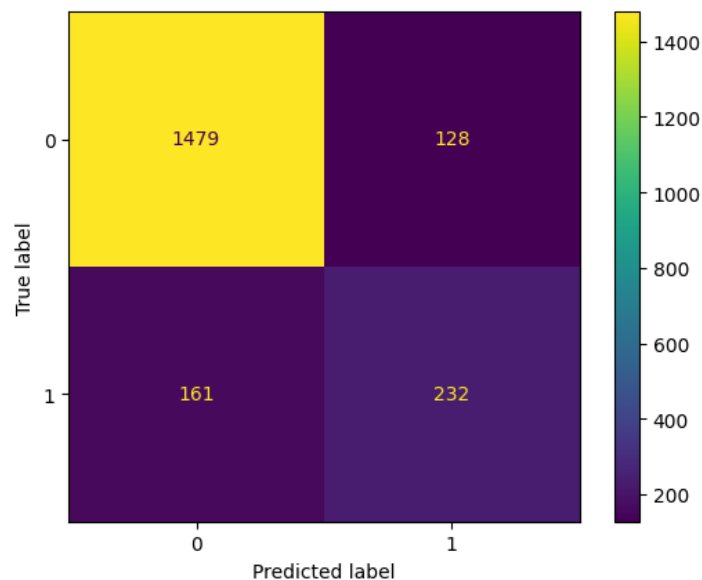


Figure 1: Confusion matrix on the 2,000-sample test set.

Training Curves

Both accuracy and loss curves show stable convergence within ~20 epochs. The validation accuracy closely tracks and occasionally exceeds training accuracy early on — a sign that Dropout regularization is working effectively to prevent overfitting.

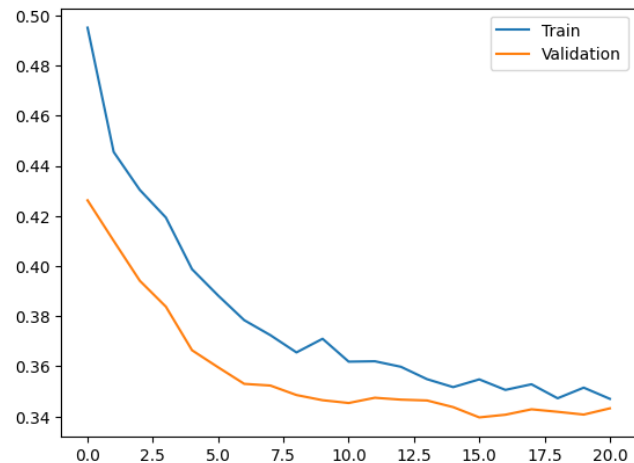
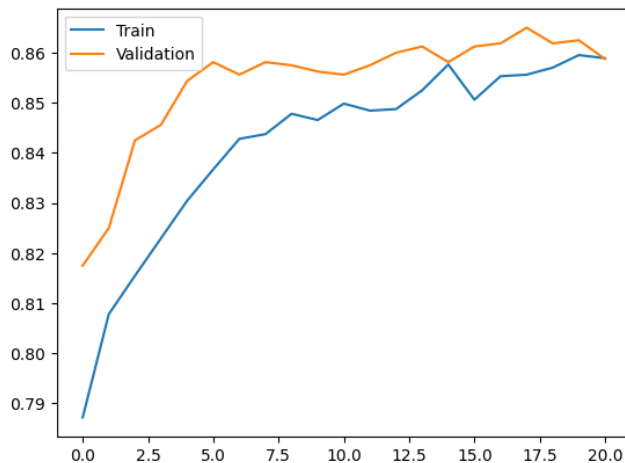


Figure 2 (left): Training vs. Validation Accuracy over epochs. Figure 3 (right): Training vs. Validation Loss over epochs.

9. Discussion

The model achieves strong performance on the majority class (stayed customers) with an F1-score of 0.91, reflecting the class imbalance in the underlying data (~80% non-churn). For the minority churn class, the model achieves a precision of 0.64 and recall of 0.59, resulting in an F1-score of 0.62 — a reasonable result given the imbalance, and improved by the lowered prediction threshold.

The three-layer Dropout architecture (0.3 → 0.2 → 0.1) and Early Stopping successfully prevent overfitting, as evidenced by the closely tracking train/validation curves. The validation accuracy briefly surpasses training accuracy in early epochs due to Dropout being active only during training, a well-known characteristic of Dropout-regularized models.

Potential Improvements

- Apply class weighting or SMOTE oversampling to better handle class imbalance
- Further threshold optimization using Precision-Recall curves
- Hyperparameter tuning (learning rate, batch size, layer widths)
- Explore ensemble methods (Random Forest, XGBoost) as baselines or stacked models
- Add Batch Normalization layers for faster and more stable convergence
- Cross-validation for more robust performance estimation

10. Conclusion

This project demonstrates the development of a Customer Churn Prediction System using a multi-layer Artificial Neural Network with TensorFlow and Keras. The model processes demographic and account-level features to predict whether a bank customer will churn, achieving an overall test accuracy of 86% with a weighted F1-score of 0.85.

Key design decisions — a progressive Dropout schedule, Early Stopping, and a custom prediction threshold of 0.35 — contribute to a robust model that generalizes well to unseen data. The project demonstrates the practical applicability of deep learning to real-world business problems in the financial sector, and establishes a strong baseline for future improvement.

References

- TensorFlow Documentation — <https://www.tensorflow.org/>
- Scikit-Learn Documentation — <https://scikit-learn.org/>
- Keras Documentation — <https://keras.io/>
- Churn Modelling Dataset — Kaggle / UCI Machine Learning Repository
- Pandas Documentation — <https://pandas.pydata.org/>